

Les mécanismes de protection

Les défenses **navigateur et HTTP** qui s'empilent : validation, CSRF, cookies, CORS, CSP, HSTS, clickjacking, sécurité des API.



Objectifs pédagogiques

À l'issue, vous saurez :

- Valider et encoder les données en **entrée** et en **sortie**.
- Protéger contre **CSRF** et sécuriser **cookies** et **sessions**.
- Maîtriser **SOP**, **CORS**, **CSP**, **HSTS** et l'anti-**clickjacking**.
- Appliquer les bonnes pratiques de **sécurité des API**.

Le principe

Ces mécanismes sont des **couches** : aucun ne suffit seul, mais **empilés**, ils constituent une défense en profondeur qui coûte cher à l'attaquant.

Beaucoup sont des en-têtes

La plupart s'activent via des **en-têtes HTTP de réponse** et des **attributs de cookie** — simples à poser, souvent oubliés.

Où agissent ces mécanismes



À retenir

Le **serveur** valide et pose les **en-têtes** ; le **navigateur** les applique (SOP, CSP, cookies). La sécurité naît de leur **coopération**.

01

Valider et encoder les données

Deux gestes distincts : filtrer ce qui entre, encoder ce qui sort.

Dans cette partie

- Validation en entrée (liste blanche, typage)
- Canonicalisation
- Encodage en sortie contextuel
- Assainissement du HTML riche

Liste blanche, typage, canonicalisation

- ✓ **Liste blanche** plutôt que liste noire (contournable)
- ✓ Valider **type, format, longueur, bornes**
- ✓ **Canonicaliser** avant de valider (Unicode, `../`)
- ✓ Valider **côté serveur** (le client ne compte pas)

```
// Bean Validation (JSR-380) – déclaratif
record Inscription(
    @Email String email,
    @Pattern(regexp="[A-Za-z -]{2,60}") String nom,
    @Min(18) @Max(120) int age) {}
@PostMapping("/inscription")
void creer(@Valid @RequestBody Inscription in) { ... }
```

Attention

Validation ≠ sécurité complète. Elle réduit la surface mais ne remplace ni les requêtes paramétrées, ni l'encodage en sortie.

Encoder selon le contexte de destination

Le principe

La même donnée s'encode **différemment** selon l'endroit où elle est écrite. C'est la parade de fond contre le **XSS**.

```
Corps HTML → &lt; &gt; &amp;  
Attribut   → encodage attribut + guillemets  
JavaScript → \xHH, ou ne pas injecter  
URL       → encodeURIComponent()
```

EN PRATIQUE

- Laisser le **moteur de template** encoder (Thymeleaf `th:text`, JSX)
- Ne pas construire du HTML par concaténation
- Pour du HTML riche : **assainir** avec une lib (OWASP Java HTML Sanitizer, DOMPurify)

02

CSRF, sessions et cookies

Empêcher un site tiers d'agir « en votre nom », et durcir les cookies.

Dans cette partie

- CSRF : mécanisme et parade
- CSRF vs XSS vs SSRF
- Cookies : Secure, HttpOnly, SameSite
- Préfixes __Host- / __Secure-
- Cycle de vie des sessions

Le navigateur envoie vos cookies... même depuis un autre site



Définition — CSRF

Le **Cross-Site Request Forgery** abuse du fait que le navigateur **joint automatiquement les cookies** à toute requête vers un domaine — même déclenchée par un **autre site**. L'action s'exécute avec votre session, à votre insu.

Un jeton anti-CSRF, imprévisible

Synchronizer token

Le serveur place un **jeton aléatoire** dans le formulaire ; il le **revérifie** à la soumission. Un site tiers ne peut pas le deviner.

```
// Spring Security : CSRF activé par défaut
http.csrf(Customizer.withDefaults());
// jeton injecté dans le formulaire :
<input type="hidden"
  name="_csrf" value="8f3a...b1">
```

✓ VARIANTES ET RÈGLES

- **Double-submit cookie** pour les API stateless
- Jeton lié à la session, à usage court
- Ne protéger que les méthodes **qui changent l'état** (POST/PUT/DELETE)
- Les API en **Bearer token** (pas de cookie) sont hors-scope CSRF

CSRF vs XSS vs SSRF

Attaque	Ce qu'elle abuse	Qui exécute	Parade de fond
XSS	Une sortie non encodée	Le navigateur (du JS injecté)	Encodage en sortie + CSP
CSRF	Les cookies auto-joints	Le navigateur (requête légitime en apparence)	Jeton anti-CSRF + SameSite
SSRF	Une URL non restreinte	Le serveur émet la requête	Liste blanche + IP résolue + cloisonnement

La clé

XSS et CSRF s'exécutent **côté client** (navigateur), SSRF **côté serveur**. Trois causes différentes, trois parades différentes — ne pas les mélanger. (SSRF relève de l'A01:2025.)

Secure, HttpOnly, SameSite

```
Set-Cookie: session=abc123;  
Secure;      // HTTPS uniquement  
HttpOnly;    // illisible en JavaScript  
SameSite=Lax; // pas envoyé en cross-site (GET top-level ok)  
Path=/; Max-Age=3600
```

- ✓ **Secure** : jamais transmis en clair (HTTP)
- ✓ **HttpOnly** : inaccessible au JS → limite l'impact XSS
- ✓ **SameSite** : bloque l'envoi cross-site → atténue le CSRF
- ✓ Préfixe `__Host-` pour verrouiller portée et Secure

Strict, Lax, None

Valeur	Envoyé en cross-site ?	Usage
Strict	Jamais	Actions sensibles ; peut casser des liens entrants (déconnecté au clic)
Lax (défaut moderne)	Seulement en navigation GET de haut niveau	Bon compromis pour un cookie de session
None	Toujours — obligatoirement avec Secure	Contextes cross-site légitimes (SSO, iframe)

Piège

`SameSite=None` sans `Secure` est **rejeté** par les navigateurs. Et `SameSite` **atténue** le CSRF mais ne le remplace pas (garder le jeton pour les cas critiques).

Les préfixes __Host- et __Secure-

Pourquoi

Un cookie peut être **écrasé** depuis un sous-domaine ou une connexion non sûre. Les **préfixes** imposent des garanties que le navigateur vérifie automatiquement.

```
Set-Cookie: __Host-session=abc;  
            Secure; Path=/; SameSite=Lax  
// __Host- exige : Secure + Path=/ + PAS de Domain
```

- ✓ **__Secure-** : le cookie doit avoir **Secure**
- ✓ **__Host-** : **Secure** + **Path=** + **pas** de **Domain** → lié à l'origine exacte
- ✓ Empêche un sous-domaine compromis d'écraser le cookie

Régénérer, expirer, invalider

- ✓ **Régénérer** l'ID de session au login (anti-fixation)
- ✓ Expiration **d'inactivité + absolue**
- ✓ **Invalider** côté serveur à la déconnexion
- ✓ Identifiant **aléatoire**, long, stocké côté serveur

```
// anti-fixation à l'authentification
request.changeSessionId();
// invalidation au logout
session.invalidate();
// config d'expiration
server.servlet.session.timeout=15m
```

03

Same-Origin Policy et CORS

La règle qui isole les origines, et le mécanisme qui l'assouplit de façon contrôlée.

Dans cette partie

- SOP : qu'est-ce qu'une « origine »
- CORS : assouplir avec l'accord du serveur
- Requête simple vs pré-vol
- Configuration CORS sûre

Same-Origin Policy : qu'est-ce qu'une « origine » ?

Origine = schéma + hôte + port

Deux pages ont la **même origine** si les trois coïncident. Sinon, le navigateur **isole** : pas de lecture croisée du DOM ni des réponses.

```
// référence : https://app.fr:443
https://app.fr/page    ✓ même origine
http://app.fr          ✗ schéma
https://api.app.fr     ✗ hôte
https://app.fr:8443    ✗ port
```

Ce que SOP protège

Empêche un site malveillant de **lire** les données d'un autre (votre webmail, votre banque) chargées dans votre navigateur. C'est la **fondation** de la sécurité Web côté client.

Nuance

SOP **bloque la lecture**, pas l'envoi : d'où la nécessité du jeton anti-CSRF pour les écritures.

Assouplir la SOP, avec l'accord du serveur cible



Définition — CORS

Le **Cross-Origin Resource Sharing** permet à un serveur d'**autoriser explicitement** certaines origines à lire ses réponses. Pour les requêtes « non simples », le navigateur envoie d'abord une requête **pré-vol** (**OPTIONS**) ; le serveur répond avec les en-têtes **Access-Control-Allow-***.

Requête « simple » vs pré-vol

Requête simple

`GET` / `POST` avec en-têtes standards et `Content-Type` de formulaire. **Pas de pré-vol** : la requête part, le navigateur vérifie juste la réponse.

Requête pré-vol (preflighted)

`PUT` / `DELETE`, **en-têtes personnalisés** ou `Content-Type: application/json` → le navigateur envoie d'abord un `OPTIONS` pour demander l'autorisation.

Conséquence pratique

Une API JSON déclenche **toujours** un pré-vol. Le serveur doit répondre correctement à `OPTIONS` (méthodes, en-têtes autorisés, `max-age` du cache de pré-vol).

Explicite, jamais grand ouvert

✗ Dangereux

```
Access-Control-Allow-Origin: *  
Access-Control-Allow-Credentials: true  
// combinaison interdite / reflète l'Origin sans filtre
```

✓ Correct

```
Access-Control-Allow-Origin: https://app.fr  
Access-Control-Allow-Methods: GET, POST  
Access-Control-Allow-Credentials: true // origine précise
```

Pièges classiques

- Refléter l'`Origin` reçue **sans liste blanche**
- `*` + credentials (rejeté, ou trou béant si contourné)
- Autoriser des origines de **dev** en prod

Rappel

CORS **autorise une lecture**, il n'« ouvre » pas l'API :
l'**authentification** et l'**autorisation** restent indispensables.

04

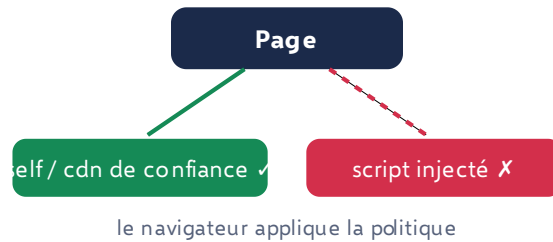
CSP, HSTS et clickjacking

Trois en-têtes qui limitent l'exécution, forcent le chiffrement et empêchent le détournement d'UI.

Dans cette partie

- CSP : principe, nonce, déploiement
- HSTS : forcer le HTTPS
- Clickjacking : mécanisme et parade
- Subresource Integrity (SRI)

N'autoriser que des sources de confiance



Définition — CSP

La **Content Security Policy** est un en-tête qui déclare les **sources autorisées** (scripts, styles, images...). Le navigateur **bloque** tout ce qui n'est pas listé — c'est une **défense en profondeur** majeure contre le XSS.

À retenir

La CSP ne remplace pas l'encodage : elle **limite les dégâts** si une XSS passe malgré tout (un script injecté n'est pas dans les sources autorisées).

Nonce, sources, remontée des violations

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-r4nd0m' 'strict-dynamic';
  object-src 'none';
  base-uri 'none';
  frame-ancestors 'none';
  report-to csp-endpoint
```

- ✓ **Nonce** par requête : seuls les scripts marqués s'exécutent
- ✓ **'strict-dynamic'** : la confiance se propage aux scripts chargés
- ✓ **Bannir** **'unsafe-inline'** et **'unsafe-eval'**
- ✓ **report-to** : recevoir les **violations**

La poser sans casser le site

✓ APPROCHE PROGRESSIVE

- Démarrer en **Report-Only** (observe sans bloquer)
- Analyser les **rapports** de violation
- Ajuster, puis **appliquer** la politique

```
Content-Security-Policy-Report-Only: ...
```

Pièges

- Une CSP à base d'**allowlist de domaines** se contourne souvent → préférer les **nonces**
- Le **scripting inline** existant demande une refonte
- CSP **complète** la prévention XSS, ne la remplace pas

HTTP Strict Transport Security

Définition

Un en-tête qui ordonne au navigateur de **toujours** utiliser HTTPS pour ce domaine pendant une durée donnée — bloquant les attaques de **rétrogradation** (SSL stripping).

```
Strict-Transport-Security:  
  max-age=63072000;      // 2 ans  
  includeSubDomains;  
  preload
```

- ✓ **max-age** élevé une fois la config stable
- ✓ **includeSubDomains** : couvre tous les sous-domaines
- ✓ **preload** : inscrit dans la liste des navigateurs (HTTPS dès la 1re visite)

Attention

preload est difficile à annuler — s'assurer que **tout** est en HTTPS avant.

Un clic détourné par une iframe invisible

Page leurre (« Gagnez ! »)

iframe transparente : bouton
« Supprimer mon compte »

Le clic va au VRAI bouton, caché

Définition

Le **clickjacking** superpose une **iframe transparente** du site cible au-dessus d'un leurre. La victime croit cliquer sur le leurre, mais son clic actionne le site cible (où elle est connectée).

La parade

Interdire au site d'être mis en cadre par un tiers → `frame-ancestors` (CSP) ou `X-Frame-Options`.

Subresource Integrity (SRI)

Définition

Le **SRI** ajoute un **hash** attendu à une ressource externe (script, feuille de style). Le navigateur **refuse de l'exécuter** si le contenu ne correspond pas — protection si un CDN est compromis.

```
<script
  src="https://cdn/lib.js"
  integrity="sha384-oqVuAf..."
  crossorigin="anonymous"></script>
// hash ≠ contenu → script bloqué
```

🛡️ **COMBINER** SRI (intégrité) + CSP (sources autorisées) pour sécuriser les scripts tiers. Rappel : intégrité = A08:2025.

Interdire d'être mis en cadre

✓ CSP moderne (recommandé)

```
Content-Security-Policy:  
  frame-ancestors 'none';  
// ou 'self' pour n'autoriser que son propre domaine
```

✓ En-tête historique (compat)

```
X-Frame-Options: DENY
```

Lequel choisir

`frame-ancestors` (CSP) est le **standard actuel**, plus souple (liste d'origines). `X-Frame-Options` reste utile pour les **vieux navigateurs**. Poser les **deux** ne coûte rien.

05

La sécurité des API et Web Services

Des risques spécifiques, un référentiel dédié : l'OWASP API Security Top 10.

Dans cette partie

- Pourquoi un référentiel à part
- GraphQL et SOAP
- Autorisation BOLA / BFLA
- Le socle d'une API sûre
- Les autres en-têtes utiles

Pourquoi un référentiel à part

Le contexte

REST, GraphQL, SOAP : les API exposent directement la logique métier, souvent sans interface. Les risques dominants sont l'**autorisation** et l'**exposition de données**.

D'où l'**OWASP API Security Top 10**, complémentaire du Top 10 « web » (vu au module 8).

Les risques n°1

- **BOLA** — autorisation au niveau objet cassée
- **BFLA** — autorisation au niveau fonction cassée
- Exposition excessive de données / propriétés
- Consommation de ressources non limitée
- Inventaire incomplet (shadow / zombie APIs)

Spécificités GraphQL et SOAP

GraphQL

- Une seule endpoint, requêtes flexibles → **DoS par profondeur/complexité**
- **Introspection** à désactiver en production
- Autorisation **par champ**, limiter la profondeur et le coût

SOAP / XML

- Risque **XXE** (XML External Entity) → RCE / SSRF / fuite
- **Désactiver** les entités externes du parseur XML
- Valider contre le **schéma** (XSD), WS-Security si besoin

Constante

Quel que soit le style (REST, GraphQL, SOAP) : **authentifier, autoriser objet et fonction, valider les entrées, limiter** les ressources.

Vérifier l'objet ET la fonction, à chaque appel

✗ BOLA — pas de contrôle d'objet

```
@GetMapping("/api/orders/{id}")
Order get(@PathVariable Long id){
    return repo.findById(id); // l'ordre d'un autre !
}
```

✓ Objet scopé + fonction protégée

```
@PreAuthorize("hasAuthority('orders:read')") // BFLA
@GetMapping("/api/orders/{id}")
Order get(@PathVariable Long id,
          @AuthenticationPrincipal User u){
    return repo.findByIdAndOwnerId(id, u.id()) // BOLA
        .orElseThrow(NotFound::new);
}
```

Le socle d'une API sûre

- ✓ **Valider le schéma** des entrées (OpenAPI / JSON Schema)
- ✓ Ne renvoyer **que** les champs nécessaires (DTO de sortie)
- ✓ **Rate limiting** et quotas (anti-abus, DoS)
- ✓ Authentification par **jeton validé** (OAuth2/OIDC, JWT)
- ✓ **HTTPS** obligatoire, versionner l'API

```
// rate limit par clé d'API / utilisateur
if (!limiter.tryConsume(apiKey))
    return status(429); // Too Many Requests
// sortie : DTO restreint, pas l'entité
record OrderView(Long id, String statut,
                  BigDecimal total) {}
```

Les autres en-têtes utiles

En-tête	Rôle
X-Content-Type-Options: nosniff	Empêche le navigateur de « deviner » le type MIME (anti-XSS via upload)
Referrer-Policy	Limite l'URL de provenance transmise (strict-origin-when-cross-origin)
Permissions-Policy	Restreint l'accès aux API navigateur (caméra, micro, géoloc)
Cross-Origin-Opener-Policy (COOP)	Isole le contexte de navigation (anti-attaques cross-origin)
Cross-Origin-Resource-Policy (CORP)	Contrôle qui peut inclure vos ressources

Approche

Poser une **base d'en-têtes** commune à toutes les applications (via le reverse proxy ou un filtre), puis affiner par application.

06

Obfuscation, minification et synthèse

Ce que ces techniques apportent... et ce qu'elles ne sont pas.

Dans cette partie

- Obfuscation et minification : portée et limites
- Les en-têtes de sécurité en un coup d'œil
- Une configuration complète
- Empiler les couches

Portée... et limites

Ce que ça fait

- **Minification** : réduire la taille du JS/CSS (performance)
- **Obfuscation** : rendre le code **difficile à lire**
- Freine la rétro-ingénierie **occasionnelle**

Ce que ça n'est PAS

- **Pas** du chiffrement : le code s'exécute, donc se lit
- **Pas** une protection des secrets (rien de secret côté client)
- « Sécurité par l'obscurité » : **ralentit**, ne protège pas

Règle

Aucune donnée ni logique de sécurité sensible ne doit reposer sur le fait que le code client est « illisible ». La sécurité se joue **côté serveur** (rappel : le client est manipulable, module 7).

Les en-têtes de sécurité en un coup d'œil

En-tête	Rôle
Content-Security-Policy	Sources autorisées → anti-XSS, anti-clickjacking (frame-ancestors)
Strict-Transport-Security	Force HTTPS (HSTS)
X-Frame-Options	Anti-clickjacking (compat)
X-Content-Type-Options: nosniff	Empêche le MIME-sniffing
Referrer-Policy	Limite les infos de provenance fuitées
Permissions-Policy	Restreint les API navigateur (caméra, géoloc...)
Set-Cookie flags	Secure · HttpOnly · SameSite

Une configuration d'en-têtes complète

nginx

```
add_header Content-Security-Policy "default-src 'self';  
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains";  
add_header X-Content-Type-Options "nosniff";  
add_header Referrer-Policy "strict-origin-when-cross-origin";
```

Spring Security 6

```
http.headers(h -> h  
    .contentSecurityPolicy(c -> c.policyDirectives(  
        "default-src 'self'; frame-ancestors 'none'"))  
    .httpStrictTransportSecurity(s -> s  
        .maxAgeInSeconds(63072000).includeSubDomains(true))  
    .contentTypeOptions(Customizer.withDefaults()));
```

Vérifier

Tester avec un scanner d'en-têtes (Mozilla Observatory, securityheaders.com) et en **DAST** (OWASP ZAP) — ces contrôles s'automatisent en CI (module 10).

Les sigles du module en un coup d'œil

CSRF — Cross-Site Request Forgery

SOP — Same-Origin Policy (isolation des origines)

HSTS — HTTP Strict Transport Security

SRI — Subresource Integrity (hash des scripts tiers)

Clickjacking — détournement de clic par iframe

DTO — Data Transfer Object (entrée/sortie dédiée)

JWT / OIDC — jeton signé / OpenID Connect

XSS — Cross-Site Scripting

CORS — Cross-Origin Resource Sharing

SameSite — attribut de cookie contre le cross-site

Nonce — valeur aléatoire à usage unique (CSP)

XXE — XML External Entity (attaque XML)

REST / GraphQL / SOAP — styles d'API / web services

DAST — test dynamique (OWASP ZAP)

SSRF — Server-Side Request Forgery (A01:2025)

CSP — Content Security Policy

__Host- / __Secure- — préfixes de cookie sécurisés

Preload — inscription HSTS dans les navigateurs

COOP / CORP — isolation cross-origin (en-têtes)

BOLA / BFLA — autorisation objet / fonction cassée (API)

Empiler les couches

- ✓ Valider en entrée, **encoder en sortie**
- ✓ Jeton anti-**CSRF** + cookies durcis (Secure/HttpOnly/SameSite)
- ✓ **SOP** isole, **CORS** assouplit avec précaution
- ✓ **CSP** (nonce) + **HSTS** + anti-clickjacking (frame-ancestors)
- ✓ API : autorisation objet/fonction, rate limit, schéma, DTO de sortie
- ✓ En-têtes complémentaires posés au reverse proxy

Et ensuite ?

Le **module 10 automatise** la vérification de tout cela (SAST/DAST/SCA, scanners d'en-têtes, quality gates), et le **module 11** traite le **durcissement système**.

L'idée à emporter

Des en-têtes simples, posés **systématiquement**, forment une défense en profondeur redoutable — chaque couche casse une étape de l'attaque.

Sources et références

Ressources

- **OWASP Cheat Sheets** : CSRF, XSS, CORS, Session, HTTP Headers
- **OWASP API Security Top 10**
- **MDN Web Docs** — SOP, CORS, CSP, cookies
- **web.dev** — sécurité navigateur (COOP/CORP, CSP)

Outils de vérification

- **Mozilla Observatory, securityheaders.com**
- **OWASP ZAP** (DAST, en CI — module 10)

À actualiser

Les comportements par défaut des navigateurs évoluent (SameSite, durée max des cookies, dépréciation de `X-XSS-Protection`). Vérifier les recommandations à jour (MDN, OWASP) avant chaque session.

Cohérence

Ce module met en œuvre les contre-mesures du module 8 (édition OWASP 2025) : BOLA/BFLA, anti-SSRF, en-têtes de sécurité.

Fin du module 9

Des questions ?

Des en-têtes simples, posés systématiquement, forment une **défense en profondeur** redoutable. Place à l'automatisation de ces contrôles.